

AIAC ASSIGNMENT - 9.5

NAME: ISHA MADEEHA

HALLTICKETNO: 2303A51017

BATCH-01

Problem 1: String Utilities Function

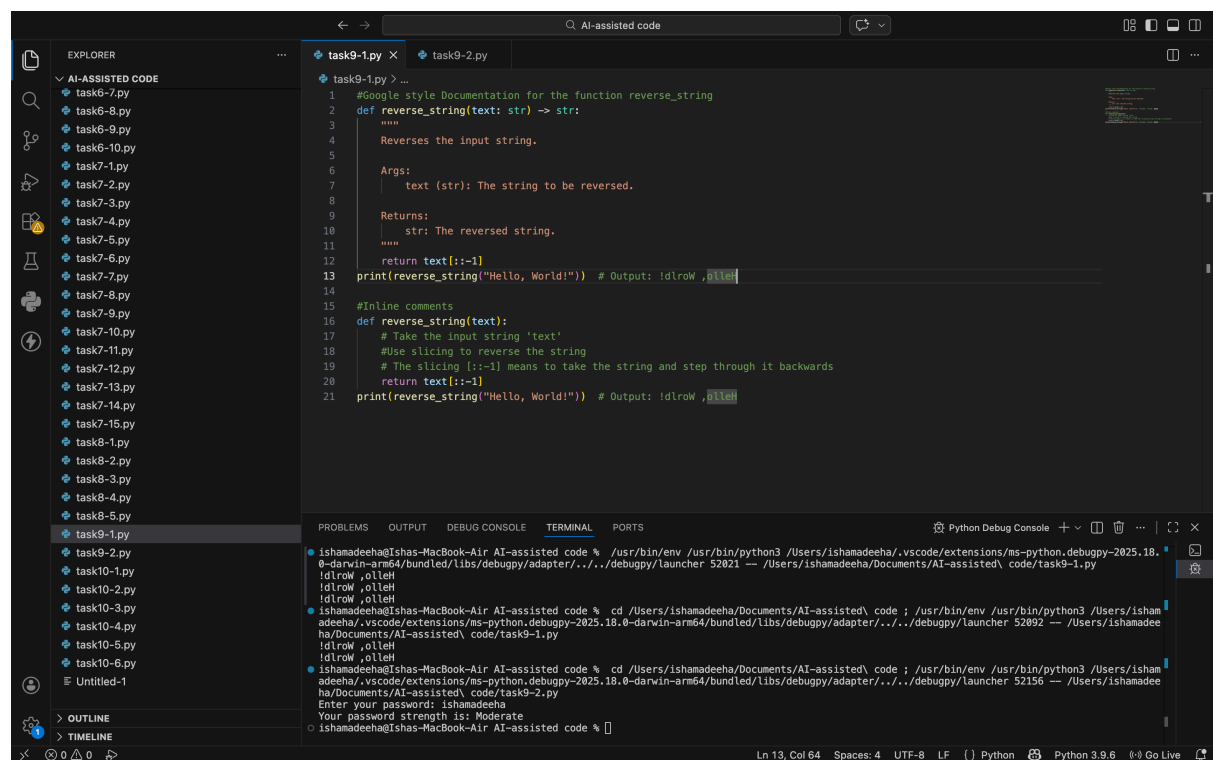
Consider the following Python function:

```
def reverse_string(text):  
  
    return text[::-1]
```

Task:

- Write documentation in:
 - Docstring
 - Inline comments
 - Google-style documentation
- Compare the three documentation styles.
- Recommend the most suitable style for a utility-based string

Library



Comparison of documentation style

Style	Purpose	Pros	Cons
Docstring (basic)	Standard internal documentation	Simple, readable, built-in help() support	Not structured for large projects
Inline comments	Explain logic step-by-step	Helpful for beginners	Clutters code if overused
Google-style	Structured documentation for teams/libraries	Professional, tool-friendly, readable, supports auto-docs	Slightly longer to write

Best Style for a Utility-Based String Library

Google-style documentation

Works well for reusable utility libraries

Supported by documentation generators (Sphinx, MkDocs, etc.)

Easy for teams to understand

Problem 2: Password Strength Checker

```
1 # it is python code that checks the strength of a password based on its length and categorizes
2 # it as Weak, Moderate, or Strong. using docstring, inline comments and function
3 # definition and google style docstring
4 def check_strength(password: str) -> str:
5     """Check the strength of a password.
6
7     Args:
8         password (str): The password to be checked.
9
10    Returns:
11        str: The strength of the password (Weak, Moderate, or Strong).
12    """
13    if len(password) < 8: # A password is considered weak if it is less than 8 characters long.
14        return "Weak"
15    elif len(password) < 12: # A password is considered moderate if it is between 8 and 12 characters long.
16        return "Moderate"
17    else:
18        return "Strong"
19 password = input("Enter your password: ")
20 strength = check_strength(password)
21 print(f"Your password strength is: {strength}") # This code checks the strength of a password based on its length and category
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Python Debug Console

```
ishamadeeha@Ishas-MacBook-Air AI-assisted code % /usr/bin/env /usr/bin/python3 /Users/ishamadeeha/.vscode/extensions/ms-python-debugpy-2025.18.0-darwin-arm64/bundled/libs/debugpy/adapter/.../debugpy/launcher 52021 -- /Users/ishamadeeha/Documents/AI-assisted/code/task9-1.py
idrow ,olleH
idrow ,olleH
idrow ,olleH
ishamadeeha@Ishas-MacBook-Air AI-assisted code % cd /Users/ishamadeeha/Documents/AI-assisted/code ; /usr/bin/env /usr/bin/python3 /Users/ishamadeeha/.vscode/extensions/ms-python-debugpy-2025.18.0-darwin-arm64/bundled/libs/debugpy/adapter/.../debugpy/launcher 52092 -- /Users/ishamadeeha/Documents/AI-assisted/code/task9-2.py
idrow ,olleH
idrow ,olleH
idrow ,olleH
ishamadeeha@Ishas-MacBook-Air AI-assisted code % cd /Users/ishamadeeha/Documents/AI-assisted/code ; /usr/bin/env /usr/bin/python3 /Users/ishamadeeha/.vscode/extensions/ms-python-debugpy-2025.18.0-darwin-arm64/bundled/libs/debugpy/adapter/.../debugpy/launcher 52156 -- /Users/ishamadeeha/Documents/AI-assisted/code/task9-2.py
Enter your password: ishamadeeha
Your password strength is: Moderate
ishamadeeha@Ishas-MacBook-Air AI-assisted code %
```

Consider the function:

```
def check_strength(password):  
    return len(password) >= 8
```

Task:

1. Document the function using docstring, inline comments, and Google style.
2. Compare documentation styles for security-related code.
3. Recommend the most appropriate style

Comparison of documentation style

Basic docstring	Medium	Explains purpose but lacks structured warnings or notes
Inline comments	Low	Good for developers but not enough for audits/reviews
Google-style	High	Supports structured notes, warnings, and future improvements

Problem 4: Attendance Management Module

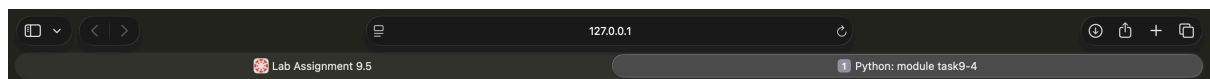
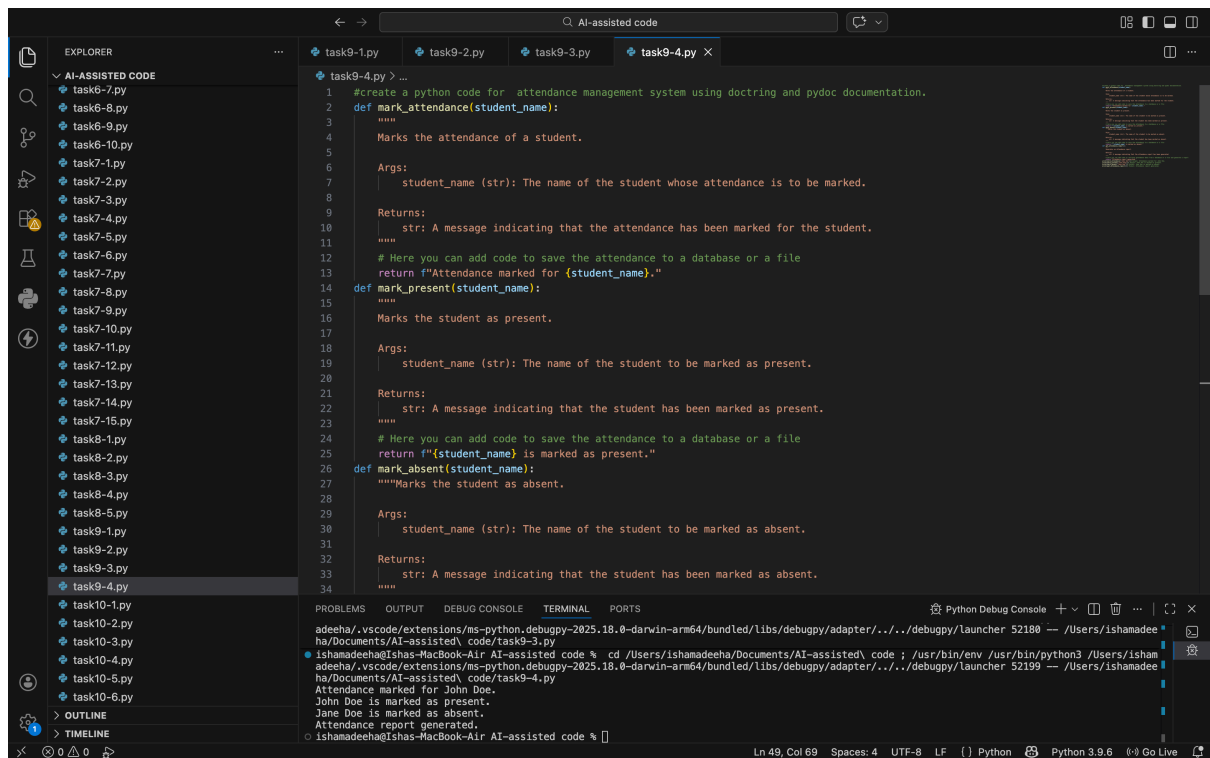
Task:

1. Create a module attendance.py with functions:

o mark_present(student)

o mark_absent(student)

o get_attendance(student)



[index](#)

[task9-4](#) [/Users/ishamadeeha/Documents/AI-assisted code/task9-4.py](#)

#create a python code for attendance management system using doctring and pydoc documentation.

Functions

get_attendance_report()

Generates an attendance report.

Returns:

str: A message indicating that the attendance report has been generated.

mark_absent(student_name)

Marks the student as absent.

Args:

student_name (str): The name of the student to be marked as absent.

Returns:

str: A message indicating that the student has been marked as absent.

mark_attendance(student_name)

Marks the attendance of a student.

Args:

student_name (str): The name of the student whose attendance is to be marked.

Returns:

str: A message indicating that the attendance has been marked for the student.

mark_present(student_name)

Marks the student as present.

Args:

student_name (str): The name of the student to be marked as present.

Returns:

str: A message indicating that the student has been marked as present.

2. Add proper docstrings.

3. Generate and view documentation in terminal and browse

Problem 5: File Handling Function

Consider the function:

```
def read_file(filename):
```

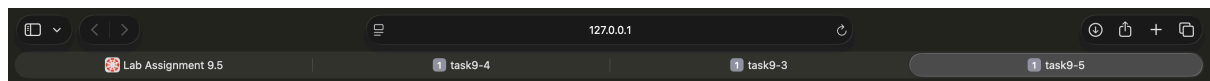
```
with open(filename, 'r') as f:
```

```
return f.read()
```

Task:

1. Write documentation using all three formats.
2. Identify which style best explains exception handling.
3. Justify your recommendation

```
11
12
13 def write_file(filename, content):
14     """Writes content to a file.
15
16     Args:
17         filename (str): The name of the file to be written to.
18         content (str): The content to be written to the file.
19
20     Returns:
21         None
22     """
23     with open(filename, 'w') as file:
24         file.write(content)
25
26 def append_file(filename, content):
27     """Appends content to a file.
28
29     Args:
30         filename (str): The name of the file to be appended to.
31         content (str): The content to be appended to the file.
32
33     Returns:
34         None
35     """
36     with open(filename, 'a') as file:
37         file.write(content)
38
39 print(read_file('attendance.txt'))# Output: Contents of attendance.txt
40 write_file('attendance.txt', 'John Doe: Present\nJane Doe: Absent\n')
41 append_file('attendance.txt', 'Alice Smith: Present\nBob Johnson: Absent\n')
42 print(read_file('attendance.txt'))# Output: Updated contents of attendance.txt
```



[index](#)
[task9-5](#) [/Users/ishamadeeha/Documents/AI-assisted code/task9-5.py](#)

#python code with three inline documentation styles: docstring, inline comments, and google style docstring for an attendance management system.

Functions

append_file(filename, content)
Appends content to a file.

Args:
 filename (str): The name of the file to be appended to.
 content (str): The content to be appended to the file.

Returns:
 None

read_file(filename)
Reads the contents of a file.

Args:
 filename (str): The name of the file to be read.

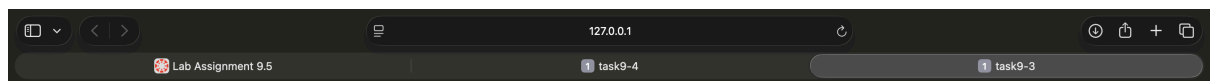
Returns:
 str: The contents of the file.

write_file(filename, content)
Writes content to a file.

Args:
 filename (str): The name of the file to be written to.
 content (str): The content to be written to the file.

Returns:
 None

PROBLEM -3:



[index](#)
[task9-3](#) [/Users/ishamadeeha/Documents/AI-assisted code/task9-3.py](#)

it is python code to define a using doc string, inline comments and function definition and
google style docstring and pytest for the function

Functions

add(a: float, b: float) -> float
Return the sum of two numbers.

Args:
 a (float): The first number.
 b (float): The second number.

Returns:
 float: The sum of a and b.

cube(x: float) -> float
Return the cube of a number.

Args:
 x (float): The number to be cubed.

Returns:
 float: The cube of x.

factorial(n: int) -> int
Return the factorial of a number.

Args:
 n (int): The number to calculate the factorial of.

Returns:
 int: The factorial of n.

square(x: float) -> float
Return the square of a number.

Args:
 x (float): The number to be squared.

Returns:
 float: The square of x.

